

Week-1 Digital-Nurture-4.0-Design Patterns:

The Reason for SOLID Principles:

design principles encourage us to create more maintainable, understandable, and flexible software. Consequently, as our applications grow in size, we can reduce their complexity and save ourselves a lot of headaches.

The following five concepts make up our SOLID principles:

1. Single Responsibility
2. Open/Closed
3. Liskov Substitution
4. Interface Segregation
5. Dependency Inversion

1->Single Responsibility

As we might expect, this principle states that **a class should only have one responsibility. Furthermore, it should only have one reason to change.**

How does this principle help us to build better software? Let's see a few of its benefits:

1. **Testing** – A class with one responsibility will have far fewer test cases.
2. **Lower coupling** – Less functionality in a single class will have fewer dependencies.
3. **Organization** – Smaller, well-organized classes are easier to search than monolithic ones.

2->Open for Extension, Closed for Modification

It's now time for the O in SOLID, known as the open-closed principle. Simply put, classes should be open for extension but closed for modification. In doing so, we stop ourselves from modifying existing code and causing potential new bugs in an otherwise happy application.

Of course, the one exception to the rule is when fixing bugs in existing code.

3->Liskov Substitution

Simply put, if class *A* is a subtype of class *B*, we should be able to replace *B* with *A* without disrupting the behavior of our program.

Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.

```
class Bird {  
    public void fly() {  
        System.out.println("Bird is flying");  
    }  
}
```

```
class Sparrow extends Bird {  
    // This works fine  
}
```

```
class Ostrich extends Bird {  
    @Override  
    public void fly() {  
        throw new UnsupportedOperationException("Ostrich can't fly");  
    }  
}
```

Problem: Ostrich is not substitutable for Bird because it violates the expectation that **all birds can fly**. This breaks LSP.

```
interface Bird {  
    void eat();  
}
```

```
interface FlyingBird extends Bird {  
    void fly();  
}
```

```
class Sparrow implements FlyingBird {  
    public void eat() {}  
    public void fly() {}  
}
```

```
class Ostrich implements Bird {  
    public void eat() {}  
}
```

4-> Interface Segregation

The I in SOLID stands for interface segregation, and it simply means that larger interfaces should be split into smaller ones. By doing so, we can ensure that implementing classes only need to be concerned about the methods that are of interest to them.

5-> Dependency Inversion

The principle of dependency inversion refers to the decoupling of software modules. This way, instead of high-level modules depending on low-level modules, both will depend on abstractions.

To demonstrate this, let's go old-school and bring to life a Windows 98 computer with code:

```
public class Windows98Machine {}Copy
```

But what good is a computer without a monitor and keyboard? Let's add one of each to our constructor so that every *Windows98Computer* we instantiate comes prepacked with a *Monitor* and a *StandardKeyboard*:

```
public class Windows98Machine {
```

```
private final StandardKeyboard keyboard;
```

```
private final Monitor monitor;
```

```
public Windows98Machine() {
```

```
    monitor = new Monitor();
```

```
    keyboard = new StandardKeyboard();
```

```
}
```

```
}
```

This code will work, and we'll be able to use the *StandardKeyboard* and *Monitor* freely within our *Windows98Computer* class.

Problem solved? Not quite. By declaring the *StandardKeyboard* and *Monitor* with the *new* keyword, we've tightly coupled these three classes together.

Not only does this make our *Windows98Computer* hard to test, but we've also lost the ability to switch out our *StandardKeyboard* class with a different one should the need arise. And we're stuck with our *Monitor* class too.

Let's decouple our machine from the *StandardKeyboard* by adding a more general *Keyboard* interface and using this in our class:

```
public interface Keyboard
```

```
public class Windows98Machine{
```

```
    private final Keyboard keyboard;
```

```
    private final Monitor monitor;
```

```
public Windows98Machine(Keyboard keyboard, Monitor monitor) {
```

```
    this.keyboard = keyboard;
```

```
        this.monitor = monitor;
    }
}
```

Here, we're using the dependency injection pattern to facilitate adding the *Keyboard* dependency into the *Windows98Machine* class.

Let's also modify our *StandardKeyboard* class to implement the *Keyboard* interface so that it's suitable for injecting into the *Windows98Machine* class:

public class StandardKeyboard implements Keyboard

Now our classes are decoupled and communicate through the *Keyboard* abstraction. If we want, we can easily switch out the type of keyboard in our machine with a different implementation of the interface. We can follow the same principle for the *Monitor* class.

Excellent! We've decoupled the dependencies and are free to test our *Windows98Machine* with whichever testing framework we choose.